

RET-Net: A CNN Framework for Real-Time Traffic Classification Using Key-Byte Mechanism

Zhe Li¹, Chengxuan Pei², Yanyue Xu, Sifan Hou, Onur Barut, Kun Qiu³, *Senior Member, IEEE*,
and Jin Zhao⁴, *Senior Member, IEEE*

Abstract—The ever-increasing diversity of encrypted traffic flows has posed significant challenges to both rule-based classification methods and traditional machine learning approaches. Recently, deep learning has emerged as a promising solution by automatically learning the features of traffic data. However, when it comes to real-time traffic classification, deep learning-based approaches often encounter performance bottlenecks. Deep learning models use complex structures to improve the accuracy of traffic classification. In addition, they typically need to extract the initial 500 to 3000 bytes of each traffic flow as input. Consequently, current deep learning-based approaches still experience milliseconds of latency in processing each traffic flow, making them unsuitable for real-world applications that demand extremely fast responses. To address this problem, we propose RET-Net. RET-Net employs the Key-Byte mechanism to identify the bytes that are decisive for traffic classification, thus significantly reducing the number of bytes required to be extracted from each flow. The extracted bytes are then processed using a refined Convolutional Neural Network architecture. Extensive experiments on real-world datasets demonstrate the remarkable performance of RET-Net. It can identify each flow in just 0.201ms. It also produces better F1 scores than state-of-the-art models while achieving approximately $21.52\times$ inference speed improvement at the same time.

Index Terms—Traffic classification, real-time, key-byte mechanism.

I. INTRODUCTION

REAL-TIME traffic classification has been the focus of network traffic analysis research [1]. In contrast to offline traffic classification, real-time traffic classification can instantly react to changes in network conditions, quickly identifying and addressing network security threats [1], [2], [3]. It also

allows for the dynamic optimization of resource allocation and Quality of Service (QoS) according to the prevailing network traffic, thus improving network performance and user experience.

Initially, real-time traffic classification was conducted using rule-based methods. As encrypted traffic has become more common, traffic information needed for classification will be encrypted and no longer visible to rule-based methods, the functionality of which will decrease as a result [4]. Meanwhile, machine learning and deep learning technologies are driving advancements in various fields. Some researchers have attempted to employ these two approaches in network traffic classification scenarios [2], [5], [6], [7].

Currently, traditional machine learning algorithms are extensively applied in real-time traffic classification, delivering satisfactory outcomes with an F1 score above 0.9 and sub-millisecond level response times [7], [8], [9]. However, these algorithms rely heavily on feature engineering, a labor-intensive process that requires significant domain-specific knowledge. Nonetheless, the ongoing evolution of network traffic poses a significant risk that the efficacy of feature engineering may diminish progressively over time.

In contrast, deep learning, with its ability to learn autonomously from raw data, has become the mainstream model for traffic classification, with F1 scores usually exceeding 0.95 [1], [10]. Despite their superior classification performance, implementing deep learning models in real-time traffic classification meets notable challenges [11], [12], [13]. The main challenge lies in the long latency introduced by the deep learning model [14]. First, state-of-the-art deep learning models usually contain complex neural networks, which need lots of computations and introduce significant latency. Moreover, these models require large amounts of data as input, for instance, 7 to 32 packets per flow, accumulating from 500 to 3000 bytes in one round of model classification [15], [16], [17]. These drawbacks result in delays of 4ms or higher [6], taking a significant part of normal internet service latency (usually from 1ms to 20ms [18], [19]), thus affecting the quality of service.

To address these challenges, we propose RET-Net, a novel, high-speed, and lightweight framework for traffic classification. RET-Net can be deployed in end systems for real-time classification of both encrypted and unencrypted traffic. There are mainly two contributions in RET-Net, including the key bytes extraction process and the lightweight Convolutional

Received 22 March 2025; revised 20 February 2026 and 19 April 2026; accepted 24 April 2026. Date of publication 30 April 2026; date of current version 6 May 2026. This work was supported by Songshan Laboratory Research Fund under grant No. ZZK202402010. The associate editor coordinating the review of this article and approving it for publication was A. Moubayed. (*Corresponding author: Jin Zhao.*)

Zhe Li and Jin Zhao are with the College of Computer Science and Artificial Intelligence, Fudan University, Shanghai 200438, China, and also with the Songshan Laboratory, Zhengzhou 452470, China (e-mail: zhelee@outlook.com; jzhao@fudan.edu.cn).

Chengxuan Pei and Kun Qiu are with the College of Computer Science and Artificial Intelligence, Fudan University, Shanghai 200438, China (e-mail: pcx15272453751@outlook.com; qkun@fudan.edu.cn).

Yanyue Xu and Sifan Hou are with Intel Asia-Pacific Research and Development Ltd., Shanghai 200241, China (e-mail: yanyue.xu@intel.com; sifan.hou@intel.com).

Onur Barut is with Intel Corporation, Santa Clara, CA 95054 USA (e-mail: onur.barut@intel.com).

Digital Object Identifier 10.1109/TNSM.2026.3689150

Neural Network (CNN). Firstly, instead of extracting the byte sequence starting from the 0-th byte of each packet, we choose to select the key bytes in the header. During this process, Key-Byte mechanism is utilized to help pinpoint the needed key bytes. By employing the Key-Byte mechanism, RET-Net only needs to extract the key bytes from the first two packets, accumulating to merely 60 ~ 70 key bytes and reducing the waiting time for more packets. Compared to other models, RET-Net needs 7 to 30 times fewer bytes. Moreover, the classifier used in RET-Net is a lightweight CNN, and it only contains two 2-dimensional convolution layers, significantly lowering computational complexity. From our experiments, RET-Net demonstrates F1 scores ranging from 0.9273 to 0.9983, and its inference speed is 20.52 times faster than other state-of-the-art deep-learning-based models. After the above two optimizations, RET-Net can finish classifying one flow within approximately 0.201ms in serial processing, regardless of the size of the network flow.

In addition, to more efficiently adapt to new traffic types, this study employs the Herding [20] algorithm to select a small set of representative samples from the old dataset, thus constructing an exemplars set. By combining the exemplars set with new traffic samples, RET-Net can utilize knowledge distillation to implement incremental learning on the existing model. Through this approach, when there is a need to recognize new traffic, it is no longer necessary to retrain on a large-scale historical dataset. Instead, the model can be fine-tuned on a smaller new dataset, significantly reducing both training time and computational costs.

In summary, we have achieved the following goals:

- We have devised an innovative Key-Byte mechanism for extracting byte sequences that are critical for classification decisions. This mechanism initially employs Gradient-Weighted Class Activation Mapping (Grad-CAM) to obtain feature matrix of different traffic types. Subsequently, the Key-Byte Region Discovery (KRD) algorithm is utilized to identify key bytes that are essential for classification decisions based on these feature matrix. Thanks to this mechanism, we can achieve high-accuracy classification tasks by extracting only 60 to 70 bytes from the first two packets of each flow. This approach significantly reduces the amount of data and computational complexity required for classification, thereby optimizing the overall classification speed.
- We have proposed an innovative framework for real-time encrypted traffic classification, named RET-Net. This framework has exhibited significant superiority in two pivotal performance metrics: classification accuracy and speed. Specifically, the RET-Net model has achieved F1 scores ranging from 0.9273 to 0.9983 across three diverse datasets. Furthermore, compared to other models, the RET-Net model has substantially increased the classification speed by a factor of 20.52, with the capability to classify an individual network flow in merely 0.201 milliseconds, irrespective of the flow's size.
- We have introduced an incremental learning mechanism into RET-Net to efficiently accommodate newly emerging traffic types. Concretely, we employ the Herding algo-

rithm to select a compact exemplars set from the old dataset, retaining only the most representative samples. Subsequently, we leverage knowledge distillation to fine-tune the existing model with a combination of these exemplars samples and new traffic data. This design enables RET-Net to rapidly adapt to new traffic without relying on large-scale retraining on the entire historical dataset, significantly reducing the time and computational resources required while effectively mitigating the issue of catastrophic forgetting.

The rest of the paper is organized as follows: Section II introduces the background of traffic classification. Section III demonstrates the overview design of RET-Net. Section IV describes the detailed architecture and working principles of RET-Net. Section V evaluates RET-Net and analyzes the evaluation results. Section VI discusses the limitations of RET-Net. Finally, Section VII concludes.

II. BACKGROUND

A. Rule-Based Methods

Initially, real-time traffic classification relied on port-based and payload-based methods [21], [22]. The port-based approach is specified by the Internet Assigned Numbers Authority (IANA) and utilizes a predefined list of ports to classify application types [23]. Nevertheless, this approach encounters several limitations [24], [25], including issues with dynamic port allocation, risks of port number exploitation by malicious users, and reduced flexibility inherent to static port-based classification systems. Payload-based approaches or deep packet inspection (DPI) techniques categorize traffic by examining packet headers and payloads [2]. However, the proliferation of encrypted traffic brings tremendous challenges to payload-based approaches [26]. They have difficulty extracting signatures from encrypted traffic, which significantly reduces their effectiveness.

B. Traditional Machine-Learning-Based Methods

With the advancement of machine learning, researchers have attempted to combine traditional machine learning algorithms with statistical features extracted from traffic data to address these challenges [2], [8], [27]. Traditional machine-learning-based methods are different from rule-based approaches by emphasizing machine-learning models, which need the development of effective feature sets [28], [29]. There are several traditional machine-learning-based works concentrating on real-time traffic classification. For instance, Pernay et al. [8] accomplished the real-time classification of media streams by utilizing meticulously optimized features derived from packet attributes and decision trees. Similarly, Dias et al. [27] also developed a classifier predicated on the Naive Bayes algorithmic framework, achieving the dual objectives of real-time video traffic classification and increased classification accuracy. Nonetheless, a notable limitation of traditional machine learning approaches is their need for sophisticated feature engineering, which requires extensive expertise, knowledge, and time. Moreover, the dynamic nature of network traffic risks rendering the features outdated.

TABLE I
SUMMARY OF REPRESENTATIVE WORKS IN ENCRYPTED TRAFFIC CLASSIFICATION

Work (Year)	Core Architecture	Input Data	Hardware	Main Limitations
Deep Packet [10] (2020)	SAE + 1D-CNN	First 1500 Bytes	-	Full input contains high redundancy.
DF [30] (2018)	CNN	5000 Packet Directions	GPU	Long sequences cause high latency.
FS-Net [4] (2019)	Bi-GRU	Full Flow Length Seq	-	Requires full flow buffering (serial bottleneck).
FlowTransformer [31] (2021)	Transformer	Flow Record Sequence	GPU	Depends on record generation; non-real-time.
TSCRNN [32] (2021)	CNN + LSTM	First 15 Packets	GPU	High memory usage; difficult for edge deployment.
GraphDApp [33] (2021)	GNN	Traffic Interaction Graph	-	High computational cost for graph construction.
ET-BERT [34] (2022)	Transformer	First 5 Packets	GPU	High compute demand; unfit for online edge inference.
MAppGraph [35] (2021)	DGCNN	Concurrent Flow Graph	GPU	Complex graph building; poor real-time performance.
CLA-BERT [36] (2025)	Transformer	256 Bytes	GPU	Dual-path architecture increases parameter count.
PACKETCLIP [37] (2025)	Hierarchical GNN	Traffic-Text Alignment	CPU	High overhead for base model loading.
FlowletFormer [38] (2025)	Transformer	First 5 Packets, 512 Bytes	GPU	Long sequence input leads to high computational cost.
RIDIT [6] (2023)	Transformer	First 2 Packets, 100 Bytes	GPU	Complex structure; high computational complexity.
MIMETIC [39] (2019)	Multi-modal CNN	12 Packets, First 576 Bytes	CPU	Still processes redundant payload data.
Synecdoche [40] (2025)	Match-Action Logic	4 Packets	Switch ASIC	Requires specialized hardware; limited flexibility.

C. Deep-Learning-Based Methods

Deep learning has established itself as the dominant paradigm in traffic classification due to its automated feature extraction capabilities [10], [41], [42], [43]. Despite achieving high precision, deploying these models on resource-constrained end systems remains challenging due to inefficiencies in data processing and model architecture.

To mitigate potential biases from manual feature selection, current approaches typically ingest full packet payloads [44]. Seminal works like Deep Packet [10] and MTL [15] process the first 1000 to 1500 bytes of a flow to maximize information retention. However, this strategy proves computationally inefficient for encrypted traffic. Since encrypted payloads exhibit high entropy and limited discriminative value compared to protocol headers, processing entire payloads consumes substantial resources on redundant data without significantly contributing to classification accuracy.

Beyond spatial redundancy, the reliance on long-range temporal dependencies introduces non-negligible buffering latency. Methods such as FS-Net [4] and FlowTransformer [31] require aggregating complete flow sequences or multiple initial packets to construct sufficient semantic context. This dependency introduces unavoidable waiting time for data accumulation, creating a critical bottleneck for real-time applications where immediate response is essential.

The escalating complexity of traffic patterns has further driven the adoption of intricate model architectures, often at the expense of inference speed. Graph Neural Networks (e.g., GraphDApp [33]) and Transformer-based pre-training models (e.g., ET-BERT [34], FlowletFormer [38]) offer stronger feature extraction but incur heavy computational overhead. Specifically, the quadratic complexity $O(N^2)$ inherent in self-attention mechanisms results in high latency on general-purpose CPUs typically found in end systems, rendering such models unsuitable for line-rate processing.

Consequently, recent research has shifted toward lightweight designs to address these deployment hurdles, yet existing solutions face distinct trade-offs [12]. Hardware-centric approaches like Synecdoche [40] achieve high throughput via ASIC

offloading but lack the flexibility to adapt to evolving application logic. Conversely, CPU-optimized models like MIMETIC [39] reduce parameter counts but do not fully address input data sparsity, nor do they provide robust mechanisms to handle the performance degradation caused by dynamic traffic evolution.

Table I summarizes the characteristics and limitations of representative works.

D. Incremental Learning

As network traffic evolves, retraining models from scratch is computationally prohibitive. Incremental Learning (IL) mitigates catastrophic forgetting when adapting to new classes. While traditional IL methods rely on basic replay or regularization, recent works have explored more sophisticated frameworks to balance plasticity and stability. For example, Zhu et al. [45] utilized generative adversarial networks to synthesize data for past classes, avoiding raw sample storage. Cerasuolo et al. [46] introduced selective smoothing and model calibration to reduce classification bias during model updates. Furthermore, Li et al. [47] applied contrastive learning to improve feature discrimination between old and new classes, while Di Monda et al. [48] adapted few-shot techniques for scenarios with limited novel traffic samples.

However, many of these advanced methods introduce structural complexities or rely on resource-intensive generative processes. The resulting computational overhead poses a practical challenge for real-time edge deployment, which typically requires strictly low latency and operates under constrained processing capabilities.

Consequently, RET-Net prioritizes computational efficiency to meet these edge constraints. Rather than deploying a structurally complex IL framework, we adopt a straightforward, lightweight strategy that combines exemplar-based replay [49] with knowledge distillation. This baseline approach provides a practical foundation for mitigating catastrophic forgetting with minimal overhead, the specific implementation details of which are expanded upon in the subsequent sections.

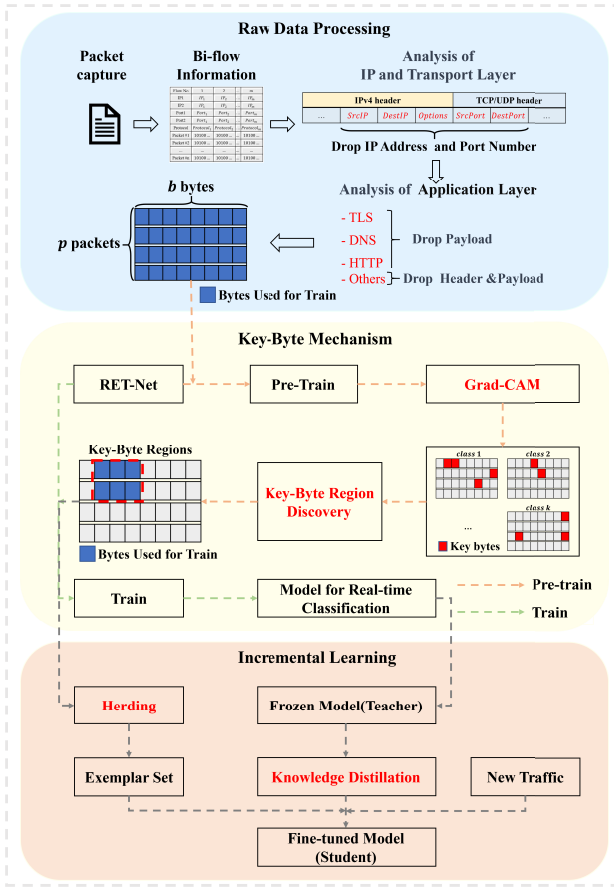


Fig. 1. The architecture of RET-Net.

III. DESIGN OVERVIEW

Fig. 1 depicts the architecture of RET-Net, comprising three core modules dedicated to the parsing and classification of network flows. A brief introduction to these essential components follows.

A. Raw Data Processing

1) *Analysis of IP and Transport Layer*: As illustrated in the top panel of Fig. 1, our process begins with the extraction of bidirectional flows from packet capture files available in public datasets. This delineation utilizes five tuples that include source and destination IPs, source and destination ports, and the transport layer protocol. For every packet in a bidirectional flow, the process needs the discarding of link-layer headers, initiating the parsing sequence from the first byte of the IP layer header. Regarding the transport layer, our analysis is confined to the UDP and TCP protocols.

2) *Analysis of Application Layer*: In handling application layer protocols, we categorize them into two groups. For TLS, HTTP, and DNS protocols, we extract header information based on domain expertise, omitting their payloads. For the remaining application layer protocols, we drop both headers and payloads. By exclusively leveraging non-sensitive header information and discarding the encrypted sensitive payload, this method averts the risk of privacy violation.

B. Key-Byte Mechanism

1) *Pre-Training Phase*: The middle panel of Fig. 1 depicts the Key-Byte Mechanism, which serves as the core feature extraction engine. After raw data processing, the raw data matrix of each flow is extracted. Then the raw data matrix is used as model input for pre-train. In the pre-train process of RET-Net, we treat packets as sequences of bytes, each represented as an integer between 0 and 255. The model commences with the extraction of semantic and positional features of bytes, facilitated by token embedding and position embedding techniques. Subsequently, a lightweight CNN is used to effectuate the classification of network traffic.

Following this, the Grad-CAM analysis was applied to the last convolutional layer of the pre-trained model to acquire the feature matrix for each type of traffic. Subsequently, we employ the KRD algorithm to identify the common key-byte region within the feature matrix of all traffic types. The region is presumed to encapsulate the most critical bytes for the model's classification decisions.

2) *Training Phase*: After the pre-training phase, the RET-Net model utilizes the data from the Key-Byte Region for formal training. Through this training process, we obtain the final model that is deployed on the endpoint system to achieve real-time traffic classification.

Key-Byte mechanism allows us to focus our attention on a finite region that is rich in feature information during the actual training. Utilizing the byte sequences within the key-byte region for training, the RET-Net model is capable of enhancing classification accuracy while concurrently reducing classification latency.

C. Incremental Learning

1) *Exemplars Set Construction*: To handle the dynamic nature of network environments, the Incremental Learning module (bottom panel of Fig. 1) enables the model to adapt without retraining from scratch. Upon receiving new traffic data, the model must integrate additional classes while preserving prior knowledge. To achieve this, we select a small set of representative samples (exemplars set) from old classes. A common approach is herding, which iteratively picks samples that best approximate the mean feature distribution of the old data. These exemplars are then combined with newly collected traffic to form an incremental training set. This strategy effectively reduces forgetting while minimizing storage overhead.

2) *Model Update via Knowledge Distillation*: Next, we employ a Teacher-Student knowledge distillation framework to retain the old model's discriminative ability for previous classes. The Teacher remains fixed, generating soft labels for old-class samples. These soft labels, containing inter-class similarity and confusion information, guide the Student model to replicate the Teacher's distribution. The overall loss is composed of a standard classification loss and a distillation loss, balanced by a weighting coefficient.

In addition, because the new categories comprise only a small proportion of the data, we refrain from modifying the key-byte regions during incremental learning to avoid

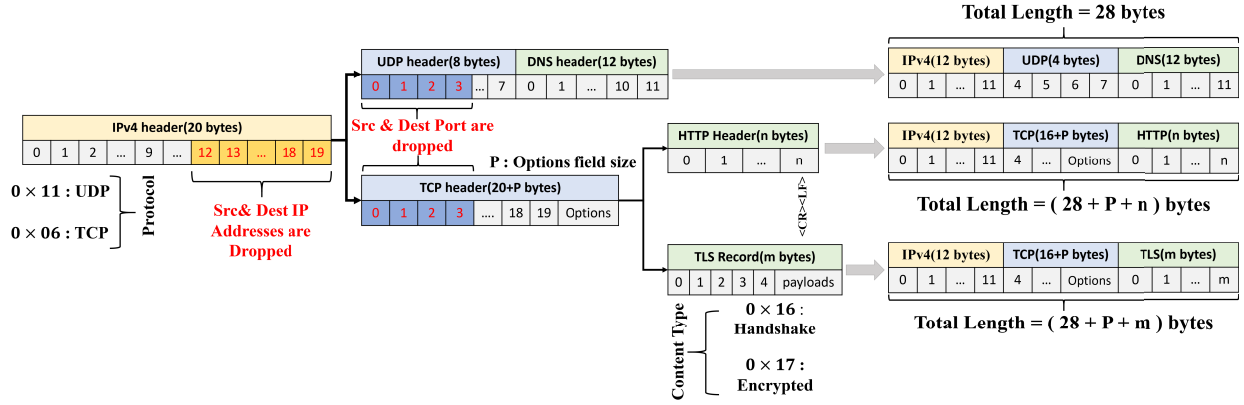


Fig. 2. Packet parsing process in RET-Net.

adversely affecting recognition of the majority of existing traffic. We only consider re-running the KRD algorithm and training a new model when the volume of new traffic substantially exceeds that of old traffic, thereby enhancing overall performance.

IV. DESIGN OF RET-NET

This section provides a comprehensive overview of our real-time traffic classification framework. Initially, we detail the methodology adopted for parsing packets. Subsequently, we present the workflow of the Key-Byte Mechanism. Finally, we will introduce the structure of the model.

A. Raw Data Processing

1) *Analysis of IP and Transport Layer:* Our system initiates packet parsing at the IP layer, extending through to the application layer headers. Fig. 2 illustrates the packet parsing process. The construction of our model leverages IPv4 network traffic, due to its prevalence in existing datasets and the real world. In the IPv4 packet header, we omit the final two segments, which encode source and destination IP addresses. This approach strengthens the model's focus on traffic classification behaviors and content features, thereby increasing the generality of the model. This approach also mitigates the risk of utilizing potentially sensitive user information. Furthermore, optional IP header fields are disregarded due to their infrequent use. Subsequent to IP header parsing, we analyze the transport layer, focusing on the TCP/UDP protocols that dominate network traffic. The initial segments of these protocols' headers, denoting source and destination ports, are similarly excluded from the analysis.

2) *Analysis of Application Layer:* The examination of the application layer concentrates on TLS, DNS, and HTTP protocols, while headers and payloads of other protocols are discarded. By employing packet headers devoid of user-sensitive information, our method prevents privacy infringement risks.

- 1) For TLS, operating over TCP for encrypted communication, we preserve the entire TLS record if the 'Content-Type' field indicates non-encrypted

user data (hexadecimal value 16). Conversely, for records containing encrypted data (hexadecimal value 17), only the TLS header is retained.

- 2) HTTP is an application layer protocol specifically designed for the exchange of data in network services. The header section of its request or response messages encompasses the essential metadata required for the classification. We identify and preserve headers up to the point marked by carriage return and line feed symbols ('' <CR><LF> ''), signifying the payload's commencement.
- 3) The DNS protocol, utilizing UDP for domain name resolution, requires preserving only the 12-byte header, excluding all payloads.

Within each network flow, all packets are parsed per a predefined method. This procedure enables the successful extraction of a raw data matrix $\mathbf{D}_{\text{raw}} \in \mathbb{R}^{p \times b}$ from each flow, where p represents the count of packets extracted from each flow and b represents the byte lengths extracted from each packet respectively. Importantly, the sequence of p extracted packets explicitly includes pure signaling packets (e.g., TCP SYN and SYN-ACK) to capture initial connection metadata.

B. Key-Byte Mechanism

The Key-Byte Mechanism primarily consists of two components: the generation of the feature matrix through Grad-CAM and the KRD algorithm. This section first outlines the features of the Key-Byte mechanism. Subsequently, it delves into the working principles of Grad-CAM and its application in traffic feature matrix extraction. Finally, the implementation details of the KRD algorithm are thoroughly described.

1) *Features:* Machine learning-based traffic classification methods typically involve a pre-model training phase where statistical features of the traffic are manually selected based on expert knowledge. The advantage of this approach is that the feature extraction process is decoupled from the training process, resulting in a model with low classification latency. However, this method is subject to the subjectivity of feature engineering and is labor-intensive. In contrast, deep learning-based traffic classification methods automatically extract features during the training process, eliminating

the need for feature engineering. The downside is that the resulting models tend to have higher computational complexity and longer classification latency.

The Key-Byte Mechanism aims to integrate the strengths of both machine learning-based and deep learning-based traffic classification methods while mitigating their respective drawbacks. Initially, the pre-training phase of the Key-Byte Mechanism can be regarded as a form of feature engineering. In this specialized feature engineering process, convolutional layers, Grad-CAM, and the KRD algorithm within the pre-trained model are utilized to automatically extract key-byte regions. Compared to traditional feature engineering, this step is devoid of human subjectivity. During the training phase, the model is formally trained using the key-byte regions extracted in the previous step to obtain a deployable model. In this step, the model ensures classification accuracy by focusing on features that are more critical for decision-making, similar to the self-attention mechanism. However, unlike the self-attention mechanism, where feature acquisition is often an integral part of the model, the pre-training phase for feature acquisition and the training phase for obtaining the deployable model in the Key-Byte Mechanism are distinct. This separation ensures that the model obtained through the Key-Byte Mechanism has extremely low computational complexity and classification latency, meeting the demands of real-time traffic classification.

2) *Grad-CAM*: Grad-CAM [50], [51] is a technique harnessed within the realm of deep learning to visualize and elucidate the decision-making processes of Convolutional Neural Networks (CNNs). This method can delineate the regions of features that contribute most significantly to the model's classification decisions.

Initially, the Grad-CAM is conventionally applied to the last convolutional layer of a CNN, as the activation maps of this layer are adept at capturing spatial information from the raw data matrix. Subsequently, the gradient with respect to the final convolutional layer is computed for the class of interest, such as the one predicted by the model. This gradient signifies the degree of contribution of each location within the feature matrix to the target class. Thereafter, a ReLU (Rectified Linear Unit) operation is performed on the gradient to ensure that all negative values are converted to zero, thereby highlighting the regions that positively contribute to the predictive outcome. Ultimately, the ReLU-processed gradient is multiplied by the corresponding feature maps, followed by a weighted averaging of all feature maps to derive an integrated feature matrix, known as the Class Activation Map. Upsampling the obtained CAM to the size of the original image yields a feature matrix of the same dimensions as the raw data matrix. Each element in the feature matrix represents the importance of the byte at the corresponding position.

Fig. 3 provides an exhaustive illustration of the Key-Byte mechanism. Initially, the raw data matrix $\mathbf{D}_{\text{raw}} \in \mathbb{R}^{p \times b}$ is selected to commence the pre-training phase, ensuring that its dimensions $p \times b$ are large enough to incorporate all potential key bytes. After the pre-training phase, the emphasis is placed on the final convolutional layer of the RET-Net model, where

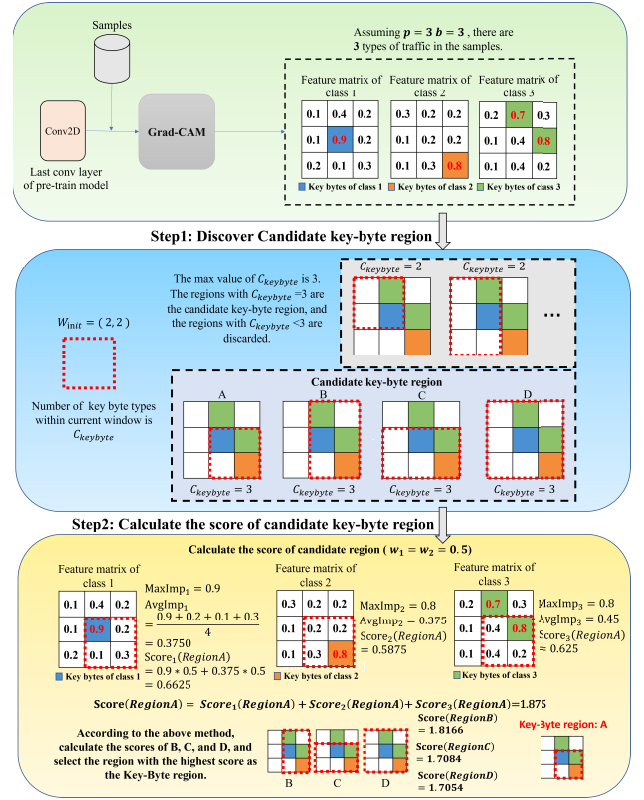


Fig. 3. Workflow of Key-Byte mechanism.

the Grad-CAM is applied to extract the feature matrix for each type of traffic within the dataset. The specific procedure is as follows:

Initially, Grad-CAM is applied to each sample to generate a corresponding feature matrix $\mathbf{F}_i \in \mathbb{R}^{p \times b}$. Subsequently, for each traffic category c within the dataset $\mathcal{S}$, the feature matrix $\mathbf{F}_i$ of all samples belonging to traffic category c are aggregated through summation to form $\mathbf{F}_{\text{agg},c} = \sum_{i \in \mathcal{S}} \mathbf{F}_i$. This step is designed to combine the common features of all samples belonging to traffic category c . $\mathbf{F}_{\text{agg},c}$ can more accurately reflect the typical characteristics of traffic category c .

To ensure numerical consistency and comparability of the feature matrix across different traffic categories, the Min-Max normalization method is employed to standardize $\mathbf{F}_{\text{agg},c}$, resulting in $\mathbf{F}_{\text{norm},c}$. This process scales the range of values within $\mathbf{F}_{\text{agg},c}$ to the interval $[0, 1]$, ensuring that the feature matrix across different traffic categories can be compared and analyzed on an equitable basis.

In $\mathbf{F}_{\text{norm},c}$, each element represents the importance of the byte in $\mathbf{D}_{\text{raw}}$ at the corresponding position. By setting a threshold τ , key bytes in $\mathbf{D}_{\text{raw}}$ can be identified. The key bytes are defined as follows using a piecewise function:

$$\mathbf{D}_{\text{key}}^c(j, k) = \begin{cases} 1 & \text{if } \mathbf{F}_{\text{agg},c}(j, k) \geq \tau \\ 0 & \text{if } \mathbf{F}_{\text{agg},c}(j, k) < \tau \end{cases} \quad (1)$$

Here, $\mathbf{D}_{\text{key}}^c(j, k)$ indicates whether the byte at position $\mathbf{D}_{\text{raw}}(j, k)$ is considered as a key byte (1) or not (0) for traffic category c . In this paper, τ is set to 0.5.

Following this, key-byte region is identified using $\mathbf{D}_{\text{key}}^c$ and $\mathbf{F}_{\text{norm},c}$ with the KRD algorithm. In the next section, the details of KRD will be presented.

3) *Key-Byte Region Discovery (KRD) Algorithm*: In the KRD algorithm, two principal stages were undertaken: the search for candidate key-byte regions and the selection of optimal key-byte regions. Initially, a dynamic sliding window-based method was developed to identify candidates for key-byte regions. The method can do adaptive adjustment of the scope of key-byte regions according to the characteristics of traffic patterns. For traffic patterns where key bytes are highly concentrated, the algorithm can correspondingly reduce the window size, thereby narrowing down the regions of key bytes to consider and enhancing efficiency. Conversely, for traffic patterns where key bytes are more sparsely distributed, the algorithm expands the window range to ensure extensive coverage of key bytes across different traffic categories, thereby enhancing the accuracy of classification.

Subsequently, to assess and select the best key-byte regions, an evaluation function was meticulously crafted. This function assigns scores to candidate regions based on a normalized feature matrix $\mathbf{F}_{\text{norm},c}$. The function integrates two pivotal metrics: the maximum importance of bytes within the region, denoted as MaxImp_c , and the average importance of all bytes, represented as AvgImp_c . The calculation for these metrics is as follows:

$$\text{MaxImp}_c = \max(\mathbf{F}_{\text{norm},c}[\text{Region}]) \quad (2)$$

$$\text{AvgImp}_c = \frac{\sum_{\mathbf{F}_{\text{norm},c}(i,j) \in \mathbf{F}_{\text{norm},c}[\text{Region}]} \mathbf{F}_{\text{norm},c}(i,j)}{|\mathbf{F}_{\text{norm},c}[\text{Region}]|} \quad (3)$$

$\mathbf{F}_{\text{norm},c}$ represents the normalized feature matrix for traffic category c . *Region* is the candidate region that we are evaluating. The $\mathbf{F}_{\text{norm},c}[\text{Region}]$ is the submatrix extracted from $\mathbf{F}_{\text{norm},c}$ corresponding to region *Region*. The $|\mathbf{F}_{\text{norm},c}[\text{Region}]|$ refers to the total number of elements within the candidate region *Region*. The evaluation function then computes the score for each candidate region by balancing the significance of the most important individual bytes with the overall average importance. This is encapsulated in the following formula:

$$\text{Score}_c(\text{Region}) = w_1 \cdot \text{MaxImp}_c + w_2 \cdot \text{AvgImp}_c \quad (4)$$

Here, w_1 and w_2 are weights that modulate the contribution of MaxImp_c and AvgImp_c to the final score, respectively. These weights are crucial as they allow us to balance the influence of the maximum importance and average importance within our scoring system.

To aggregate contributions from all traffic categories, we sum the individual scores using Eq. 5. Crucially, this evaluation function inherently regularizes heterogeneous traffic distributions. Excessive window expansion to capture dispersed features simultaneously ingests task-irrelevant bytes, diluting feature density and decreasing the AvgImp_c metric. By penalizing overly broad windows, this structural constraint encourages a compact balance between comprehensive feature coverage and a high density of informative features.

$$\text{Score}(\text{Region}) = \sum_{c \in S} \text{Score}_c(\text{Region}) \quad (5)$$

$\text{Score}(\text{Region})$ represents the final score of candidate region *Region* across all traffic categories.

This approach takes into account both the most important bytes and the average importance of all bytes within the candidate region for each traffic category. In this manner, the evaluation function can achieve a balance between the significance of individual bytes and the overall average importance. This balance is instrumental in circumventing the selection of key-byte regions that are either too localized or too diffuse, enhancing both the accuracy and robustness of the classification process. This holistic approach ensures that the selected key-byte regions are effective not only for specific traffic categories but also applicable to the entire dataset.

The KRD algorithm encompasses the following detailed steps:

- **Definition of Window Size Boundaries**: The minimum window size is set to $W_{\min} = (h_{\min}, w_{\min})$, where $h_{\min}$ is the minimum height and $w_{\min}$ is the minimum width of the window. The maximum window size is set to $W_{\max} = (h_{\max}, w_{\max})$, where $h_{\max}$ is the maximum height and $w_{\max}$ is the maximum width of the window.
- **Initialization of Window**: The initial window W_{init} is set to the minimum size $W_{\min}$, that is, $W_{\text{init}} = (h_{\min}, w_{\min})$.
- **Sliding Window Process**: Simultaneously across all $\mathbf{D}_{\text{key}}^c$, a sliding window is initiated. The window commences its traversal from the upper left corner of the matrix, proceeding in a row-major order. In this process, the sliding window should remain within the confines of the $\mathbf{D}_{\text{key}}^c$ boundaries.
- **Key Byte Distribution**: At each window position, the distribution of key bytes within the window is evaluated to determine how many distinct traffic types contain key bytes within the window's area.
- **Window Size Adjustment**: The window size is incrementally increased according to a predetermined step until it reaches $W_{\max}$. After each increment, the sliding window process and key byte distribution evaluation are repeated.
- **Recording Candidate Regions**: Among all window positions, the ones that cover the most traffic categories are selected as candidate key-byte regions, and their position and size are recorded.
- **Candidate Regions Evaluation**: For each candidate region, denoted as *region*, the evaluation function is employed to obtain its $\text{Score}(\text{Region})$. The regions are then ranked in descending order of their scores. The region with the highest score is selected as the optimal key-byte region.

C. RET-Net Model

Fig. 4 illustrates the RET-Net model. The raw data matrix or the Key-Byte region serves as its input. In our model, each byte within a packet is considered a word within a sentence in the domain of text analysis. This view is predicated upon the observed similarities between byte sequences in packets and sentences in textual content. Initially, parallelism in positional characteristics is identified: both sentences in text

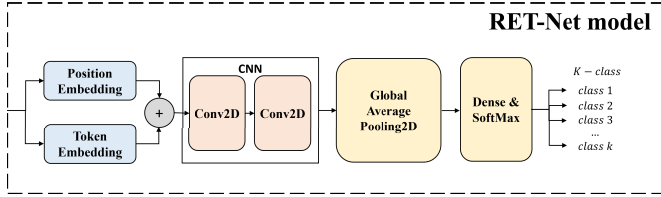


Fig. 4. Architecture of the RET-Net model.

and packet formats are governed by stringent rules that dictate the potential for the meaning of a word or byte to be altered based on its positional context. Furthermore, a congruence in semantic features is noted: both words and bytes are full of distinct semantic information, which is shaped by their literal meaning and the context of their usage. Capitalizing on these similarities, token embedding and position embedding techniques are utilized, as illustrated in Fig. 4, to delineate the semantic and positional attributes inherent in byte sequences.

Following the embedding phase, the resulting data is fed into a lightweight CNN model. The use of a CNN offers two main advantages. Firstly, the architecture of CNN enables highly efficient parallel processing, which significantly enhances computational speed compared to more complex models like Transformers. Secondly, CNN is adept at identifying and extracting local patterns through their convolutional layers, an attribute that is particularly beneficial when local semantic features are crucial for the task. As demonstrated in the experiments discussed in Section V, the bytes critical to classification decision tend to cluster together, highlighting the suitability of CNN for this application.

We use Cross Entropy Loss as the loss function. The formula for Cross Entropy Loss in multi-class classification is delineated as:

$$L_{CE} = - \sum_{c=1}^M y_{o,c} \log(p_{o,c}) \quad (6)$$

L_{CE} represents the computed loss value. M denotes the total number of classes. $y_{o,c}$ is a binary indicator (0 or 1) that signifies whether class c is the correct classification for observation o . $p_{o,c}$ is the predicted probability that observation o belongs to class c .

D. Incremental Learning

In network environments, traffic types are constantly evolving. When new traffic types emerge, the RET-Net model must be updated to handle these changes. In traditional methods, each update to the model requires retraining with the complete set of old data plus the new traffic data, along with running the KRD algorithm. However, this process is costly. To make updates more efficient, we adopt an incremental learning approach for model adjustment.

Incremental Learning refers to a training paradigm in which a model continually receives new data and tasks and updates itself accordingly. Unlike traditional batch learning, the main objective of incremental learning is to integrate new information without fully retraining the entire model,

while also avoiding—or at least reducing—the phenomenon of “catastrophic forgetting” of previously acquired knowledge. As a result, incremental learning is well-suited for scenarios where all training data cannot be collected at once or when data continues to be generated over time.

In this study, incremental learning chiefly consists of two stages: (1) constructing a representative exemplars set, and (2) adjusting the model through knowledge distillation.

1) *Construct Exemplars Set*: In the exemplars set construction stage, once new training data becomes available, we select a small number of samples from historical data that are most representative or have the greatest impact on the model’s performance. These selected samples form a small “replay” set, known as the exemplars set. Then, both these historical samples and newly collected traffic samples are used together for incremental training. Through this process, the model can “recall” previously acquired knowledge while using only a minimal amount of historical data, thereby greatly reducing the risk of forgetting.

Algorithm 1 Exemplars Selection via Herding

Require:

- 1: D_c : Samples of class c (input data)
- 2: $f(\cdot)$: Feature extractor (e.g., model backbone)
- 3: m : Number of exemplars to select

Ensure:

- 4: S_c : Selected exemplars for class c (size m)
 - 5: **function** BUILD_EXEMPLARS(D_c, f, m)
 - 6: Extract features: $\Phi_c = \{f(x) \mid x \in D_c\} \triangleright \Phi_c \in \mathbb{R}^{|D_c| \times d}$
 - 7: Compute class mean: $\mu_c = \frac{1}{|D_c|} \sum_{\phi \in \Phi_c} \phi \triangleright \mu_c \in \mathbb{R}^d$
 - 8: Initialize exemplars set: $S_c \leftarrow \emptyset$
 - 9: Initialize exemplars features: $E \leftarrow \emptyset$
 - 10: **for** $k = 1$ **to** m **do**
 - 11: **if** $k = 1$ **then**
 - 12: Target: $\mu_{\text{target}} \leftarrow \mu_c$
 - 13: **else**
 - 14: Current mean of E : $\mu_E = \frac{1}{k-1} \sum_{\phi \in E} \phi$
 - 15: Target: $\mu_{\text{target}} \leftarrow \mu_c \cdot k - \mu_E \cdot (k-1)$
 - 16: **end if**
 - 17: Compute distances: $d_i = \|\phi_i - \mu_{\text{target}}\|_2^2, \forall \phi_i \in \Phi_c$
 - 18: Mask already selected indices: $d_i \leftarrow \infty, \forall i \in S_c$
 - 19: Select closest sample: $i^* = \arg \min_i d_i$
 - 20: Add to exemplars: $S_c \leftarrow S_c \cup \{x_{i^*}\}$
 - 21: Update exemplar features: $E \leftarrow E \cup \{\phi_{i^*}\}$
 - 22: **end for**
 - 23: **return** S_c
 - 24: **end function**
-

To choose the representative samples from the old data, we adopt the herding algorithm to select instances that best approximate the mean feature distribution. The detailed procedure is explicitly described in Algorithm 1.

The process begins by taking the samples of a specific class D_c , the feature extractor $f(\cdot)$, and the target number of exemplars m as inputs. First, the algorithm extracts the feature representations Φ_c for all samples and computes the global class mean μ_c , which serves as the statistical center of the

class (Line 6-7). The selection then proceeds iteratively for m steps (Line 10-22).

In each iteration k , we determine a dynamic target vector μ_{target} to guide the selection. For the first exemplar, the target is simply the global mean μ_c ; for subsequent steps, the target is adjusted based on the current mean of the already selected exemplars μ_E (Line 11-16). Specifically, the update rule $\mu_{target} \leftarrow k \cdot \mu_c - (k-1) \cdot \mu_E$ is designed to pull the average of the selected set closer to the true global mean with each addition. Next, the algorithm calculates the Euclidean distance between each candidate sample's feature vector and this target (Line 17). The sample that minimizes this distance is identified (Line 19) and added to the exemplar set S_c (Line 20-21). This iterative greedy optimization enables the final exemplar set S_c to effectively approximate the mean feature representation of the original data.

Upon completion of the selection process, we merge the newly constructed exemplar set with the new traffic data to form an incremental training set. This allows the model to adapt to new types of traffic while substantially preserving its prior discriminative capabilities, thereby mitigating catastrophic forgetting of old knowledge.

2) *Model Adjustment Through Knowledge Distillation:* Knowledge distillation was first proposed by Hinton et al [52]. It transfers the knowledge of an existing, already-trained model (the Teacher) to a smaller or new model (the Student). The key idea is to use the “soft labels” produced by the Teacher—these labels reflect how confident the Teacher is about each class—to guide the Student's training. Compared with the “hard labels” (true labels), soft labels often provide richer information about the relative relationships among classes.

a) *Freezing the Old Model (Teacher):* The old model, trained in the previous phase, usually has strong recognition abilities for old classes. To preserve this capability, the old model should remain unchanged during the current stage and only provide soft labels. Hence, all its parameters are fixed and not updated.

b) *Training the New Model (Student) with Soft Labels:* In the current phase, the new model must learn both new classes and old classes. For old classes, besides keeping a small number of representative samples (exemplars), we also obtain the old model's output distribution—i.e., soft labels—for these old-class samples. Soft labels, which reveal similarities or potential confusions among classes, offer more information than hard labels (only class indices or one-hot vectors). By imitating this distribution, the new model helps retain its ability to distinguish old classes.

c) *Adding a “Knowledge Distillation” Term to the Loss Function:* When training the new model, the overall loss consists of:

- **A regular classification loss:** Ensures the new model can correctly classify both new and old classes. This is implemented using the **Cross-Entropy Loss**.
- **A knowledge distillation loss:** Encourages the new model's output distribution over old classes to match the old model's predictions. This is formulated using the **KL**

divergence:

$$L_{\text{Distill}} = \frac{1}{N} \sum_{i=1}^N D_{\text{KL}}(\tilde{\mathbf{p}}^{(i)} \parallel \tilde{\mathbf{q}}^{(i)}), \quad (7)$$

where:

- $\tilde{\mathbf{p}}^{(i)} = \text{softmax}(\mathbf{t}^{(i)})$ is the teacher's probability distribution over old classes ($\mathcal{C}_{\text{old}}$) for sample i ,
- $\tilde{\mathbf{q}}^{(i)} = \text{softmax}(\mathbf{s}^{(i)})$ is the student's distribution over old classes,
- $\mathbf{t}^{(i)}, \mathbf{s}^{(i)}$ are logits from the teacher and student models, respectively.

For old-class samples, the Student predicts a distribution for all classes, while the Teacher provides its distribution specifically for the old classes. Using the KL divergence, the difference between the Student's and Teacher's distributions on these old classes is calculated and added as a distillation loss to the total loss.

d) *Balancing New and Old Knowledge:* Relying only on distillation loss might leave the new model undertrained on the new classes. Relying only on the regular classification loss, however, might cause the model to forget old knowledge. Therefore, a weighting coefficient is used to balance the distillation loss and classification loss, so that the model retains adequate performance on both new and old classes.

The total loss combines these two components:

$$L_{\text{total}} = L_{\text{CE}} + \lambda \cdot L_{\text{Distill}}, \quad (8)$$

where λ controls the trade-off between learning new classes and retaining old knowledge.

e) *Final Outcome:* By training in this manner, the new model gains sufficient discriminative power for the new classes while also inheriting the old model's output distribution for old classes, thereby reducing forgetting. The additional use of exemplars replay further helps the model see part of the original old-class samples during training, making the distillation loss more effective for these old classes.

f) *Keep the key byte region unchanged:* During this training stage, we do not modify the key byte region for two main reasons. First, if the key bytes of the new traffic do fall within this region, keeping it unchanged is reasonable for recognizing that new traffic. If the new traffic's key bytes lie outside of it, this fact itself becomes a helpful feature that allows the model to distinguish new from old traffic. The model can learn that “the new and old traffic differ in their key byte distributions.” Second, in our scenario, newly emerging traffic usually makes up only a small fraction of the overall data, meaning old traffic still dominates. Frequently making large adjustments to the key byte region could undermine existing feature extraction capabilities, leading to poorer recognition performance for most old traffic. Therefore, when new traffic is relatively small and old traffic still dominates, leaving the key byte region unaltered helps avoid major distribution shifts.

If we wished to dynamically change the key byte region as new classes appear, we would need an additional mechanism to determine “when” and “under what criteria” to make these changes, which would increase both system complexity and maintenance costs. By contrast, keeping the key byte region

fixed simplifies the incremental learning process and places more emphasis on modeling how the new traffic differs from the old. However, if over time the new traffic grows so large that it becomes the majority (an extreme case), reapplying the KRD algorithm on an entirely fresh dataset would be a better choice than continuing incremental adjustments or training a new model directly.

V. EVALUATION

A. Datasets

We utilized three publicly accessible datasets, summarized in Table II, to evaluate RET-Net under diverse conditions spanning traffic visibility, threat scenarios, and statistical distribution.

- **Traffic Visibility:** NetML primarily consists of encrypted TLS flows to challenge payload-independent analysis. In contrast, CICIDS2017 and CICDDOS2019 contain mixed protocols, testing robustness across both encrypted and transparent traffic.
- **Threat Scenarios:** The datasets cover a broad security spectrum, including heterogeneous intrusions in CICIDS2017, volumetric DDoS saturation in CICDDOS2019, and fine-grained malware classification in NetML.
- **Statistical Distribution:** To stress-test adaptability to class imbalances, we selected the benign-dominant CICIDS2017 for background noise filtering, the attack-dominant CICDDOS2019 for simulating active assaults, and the long-tail NetML dataset for evaluating few-shot capabilities on scarce minority classes.

To ensure robustness and generalizability, a ten-fold cross-validation method was adopted across all datasets.

B. Baselines

To systematically evaluate RET-Net across diverse technical paradigms, the following baseline models were selected:

- 1) **Random Forest (RF):** As a representative ensemble learning algorithm, RF was selected to benchmark against traditional methods that typically rely on labor-intensive feature engineering. This comparison is intended to demonstrate RET-Net's capacity for fully automated feature extraction. In our setup, the RF model utilizes 100 decision trees with a maximum depth of 20.
- 2) **K-Nearest Neighbors (KNN):** KNN is a non-parametric algorithm that predicts labels based on distance metrics. Similar to RF, it serves as a baseline to evaluate the trade-off between the computational simplicity of traditional machine learning and the representational capacity of deep learning approaches. The number of neighbors was set to 5.
- 3) **1D-CNN:** This model was selected as a structurally similar baseline to control for the architectural variable. Unlike RET-Net, standard 1D-CNNs typically require substantially larger input sequences (784 bytes) to extract effective features [56]. This comparison is

TABLE II
DETAILS OF THE DATASETS

#	Dataset	Category	Total Samples
1	NetML [53]	20	288918
2	CICDDOS2019 [54]	13	388095
3	CICIDS2017 [55]	7	452705

TABLE III
EXPERIMENTAL ENVIRONMENT

Component	Specification
CPU	Intel Core i5-13600KF
GPU	NVIDIA GTX 4080
OS	ubuntu 22.04
Python	3.10.13
PyTorch	2.2.1
CUDA	12.1

designed to assess the efficiency of the proposed Key-Byte mechanism in achieving robust performance with a highly compacted input of 60-70 bytes.

- 4) **Residual 1-D Image Transformer (R1DIT):** As a representative Transformer-based traffic classification model [6], R1DIT leverages multi-head self-attention mechanisms. Notably, it also performs coarse-grained feature selection by focusing on specific packet segments. We selected this baseline to benchmark our fine-grained Key-Byte mechanism against a coarse-grained strategy within a distinct architectural paradigm.

C. Experimental Setup

To accurately measure the classification prowess of the system we devised, we engaged a comprehensive set of standardized metrics, which include Accuracy, Precision, Recall, and Macro F1 Score.

Our experimental environment is presented in Table III. The training process is performed on a GPU, while the statistical inference time is calculated on a mid-to-low-end CPU from an earlier generation. This illustrates that our experimental results do not rely on high-performance equipment.

D. Key-Byte Mechanism

1) **Pre-Training Phase:** In this experimental phase, the raw data matrix D_{raw} of size $p = 2$ and $b = 125$ was selected for pre-training the model. This implies that only the first two packets of each flow are parsed, with 125 bytes extracted from each packet.

In real-time traffic classification, the maximum value for p can be 2. In this setting, RET-Net can extract a sufficient sequence of bytes from the first packet of the sender and the corresponding response packet of the receiver. It allows the receiver to classify the flow immediately upon receiving the sender's initial packet, without awaiting additional subsequent packets. Should p be set to a value greater than 2, the recipient would be required to wait for a sufficient number of packets to classify the bidirectional flow. It will lead to prolonged waiting

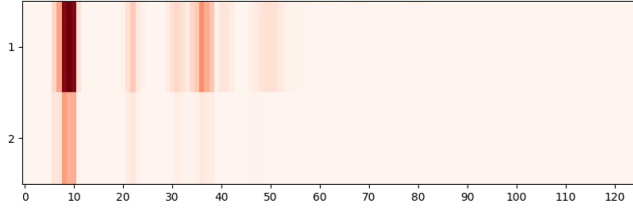


Fig. 5. Heatmap of traffic labeled 'benign' in CICIDS2017.

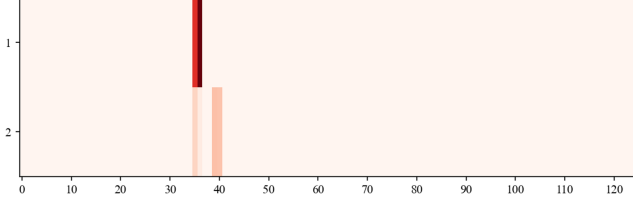


Fig. 6. Heatmap of traffic labeled 'ssh-patator' in CICIDS2017.

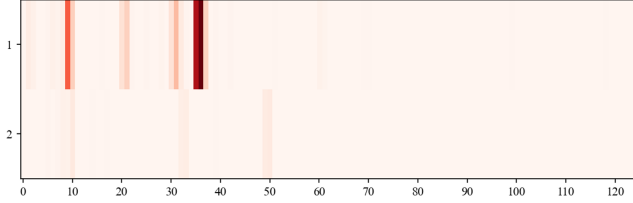


Fig. 7. Heatmap of traffic labeled 'adload' in NetML dataset.

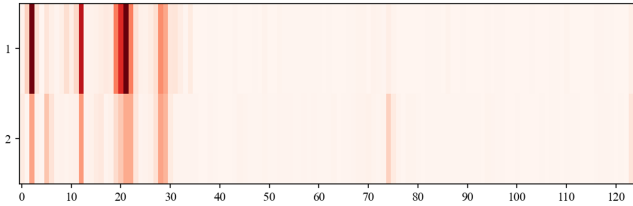


Fig. 8. Heatmap of traffic labeled 'DrDoS_UDP' in CICDDOS2019.

times when the number of packets exchanged in the network flow is less than p . Thus the setting ($p > 2$) is impractical for traffic classification that demands real-time processing. The configuration of $b = 125$ is chosen to ensure that the raw data matrix encompasses as many potential key bytes as possible.

2) *Training Phase*: Following the pre-training phase of our model, we employed Grad-CAM to produce the feature matrix for each traffic category. To provide a direct representation of Grad-CAM's effectiveness, the feature matrix is displayed as a heatmap. Figs. 5~6 illustrate the heatmaps for 'benign' and 'ssh-patator' traffic from the CICIDS2017. It is observable that for the 'benign' traffic, the key bytes are predominantly located within the first 10 bytes of the IP header, which is a part of the header checksum. For 'ssh-patator' traffic, the critical bytes are focused around the 38th byte. Furthermore, Fig. 7 presents the heatmap for 'adload' traffic in the NetML dataset, showing discriminative features concentrated in the specific fields of the application layer. Finally, Fig. 8 depicts the 'DrDoS_UDP' traffic in CICDDOS2019, where high-importance bytes are densely distributed across the IP and UDP headers, aligning with the structural characteristics of volumetric attacks.

TABLE IV
THE RESULT OF KEY-BYTE REGION DISCOVERY

Dataset	Range _{packet}	Range _{byte}	Count _{byte}
NetML	[0,2)	[8,38)	60
CICDDOS2019	[0,2)	[3,37)	68
CICIDS2017	[0,2)	[8,40)	64

Subsequently, we employ the KRD algorithm to identify key-byte regions within each dataset. In our experiments, we initially define the size boundaries of the window: the height is set to 2, with the width ranging from 30 to 50. Additionally, both weight parameters, w_1 and w_2 , within the evaluation function are assigned a value of 0.5. We then initiate the dynamic sliding window to search for candidate key-byte regions. Ultimately, the evaluation function is applied to each candidate region to ascertain its score, which is subsequently ranked according to its magnitude. The region with the highest score is designated as the final key-byte region.

Table IV presents the outcomes of the KRD algorithm. For the NetML, the key-byte region corresponds to the submatrix $D_{\text{raw}}[0 : 2, 8 : 38]$ of D_{raw} , encompassing a total of 60 bytes. In the case of the CICDDOS2019, the key-byte region is mapped to the submatrix $D_{\text{raw}}[0 : 2, 3 : 38]$, which contains 68 bytes in total. For the CICIDS2017, the key-byte region corresponds to the submatrix $D_{\text{raw}}[0 : 2, 8 : 40]$, totaling 64 bytes. Following the identification of key-byte regions, these regions are utilized as inputs to train classification models. From the coverage of the key-byte regions, it can be observed that the key bytes exhibit a feature of local convergence within the first 50 bytes.

E. Parameter Sensitivity Analysis

To verify the robustness of KRD hyperparameters, we analyze the window size and weight parameters.

First, the window size boundaries are determined by network protocol structures. We set the minimum width $W_{\min} = 30$ bytes to cover essential IPv4 and Transport Layer control fields (e.g., Protocol, Flags), which typically occupy roughly 28 bytes. The maximum width $W_{\max}$ is limited to 50 bytes to focus on local feature concentrations.

Second, we performed a sensitivity analysis on the weight parameter w_1 (range [0.3, 0.7]) in Eq. 4, with w_2 constrained as $1 - w_1$. As shown in Fig. 9, the Intersection over Union (IoU) scores remain consistently high across datasets. This stability, driven by the rigid positional characteristics of network protocol headers, demonstrates that the algorithm is highly insensitive to minor weight variations. Given this robustness, we opted to equally balance the peak feature value and the average regional density by setting $w_1 = w_2 = 0.5$. This configuration yields reliable performance across datasets without requiring complex parameter tuning.

F. Comparison of Performance With and Without KRD

To evaluate the effectiveness of the KRD algorithm, we compare RET-Net_{KRD} against two baselines: one

TABLE V
COMPARISON BETWEEN RET-NET MODELS WITH AND WITHOUT KRD

Model	NetML		CICDDOS2019		CICIDS2017	
	F1 Score	Region _{train}	F1 Score	Region _{train}	F1 Score	Region _{train}
RET-Net _{KRD}	0.9637	$D_{\text{raw}}[0 : 2, 8 : 38]$	0.9273	$D_{\text{raw}}[0 : 2, 3 : 37]$	0.9983	$D_{\text{raw}}[0 : 2, 8 : 40]$
RET-Net _{WithoutKRD}	0.9045	$D_{\text{raw}}[0 : 2, 0 : 30]$	0.9271	$D_{\text{raw}}[0 : 2, 0 : 34]$	0.5372	$D_{\text{raw}}[0 : 2, 0 : 32]$
RET-Net _{Random}	0.4026	-	0.5631	-	0.3829	-
Improvement_{KRD} vs WithoutKRD	6.55%	-	0.02%	-	85.8%	-

extracting sequences sequentially from the zeroth byte (RET-Net_{WithoutKRD}) and another using random extraction (RET-Net_{Random}). As detailed in Table V, KRD's performance gains relative to the static zeroth-byte baseline vary significantly. This variation is closely associated with the spatial distribution of discriminative features across network protocols. The adaptive capacity of KRD allows it to dynamically align with these diverse distributions, whereas static extraction windows frequently prove insufficient.

On the CICIDS2017 dataset, RET-Net_{KRD} achieves a substantial 85.8% F1 score improvement over RET-Net_{WithoutKRD}. This gain can be attributed to the fact that highly discriminative features for specific attacks reside beyond the default $[0, 32]$ byte range. As illustrated by Fig. 6, these features are primarily concentrated between bytes 35 and 40. By dynamically shifting the window to span $[8, 40]$ bytes, KRD captures displaced features that static methods typically discard.

For the NetML dataset, KRD yields a 6.55% improvement. This dataset comprises diverse malware families characterized by spatially dispersed features, as seen in the adload heatmap (Fig. 7). While the static $[0, 30]$ byte window captures a portion of the relevant information, it simultaneously ingests task-irrelevant fields at the packet's immediate beginning (e.g., initial lower-layer headers). By shifting the extraction region to $[8, 38]$ bytes, KRD bypasses these initial non-discriminative bytes and extends the trailing boundary, thereby increasing the density of informative features within the input matrix.

Conversely, the improvement on CICDDOS2019 is marginal (0.02%), reflecting a scenario where the default extraction strategy already establishes a strong baseline. This dataset predominantly contains volumetric DDoS attacks, whose distinguishing characteristics consistently reside within standard protocol headers at the packet's immediate beginning (Fig. 8). Consequently, the KRD-identified region ($[3, 37]$ bytes) heavily overlaps with the default window ($[0, 34]$ bytes). Under these homogeneous traffic conditions, the primary discriminative features are already effectively encompassed by the static baseline.

Synthesizing the results from these diverse environments, we observe that KRD's advantage is most pronounced when the default extraction strategy is misaligned with the spatial feature distribution. Specifically, the KRD algorithm provides notable benefits under two distinct conditions: (1) when traffic exhibits a significant feature offset, placing discriminative characteristics in variable fields, deep headers, or

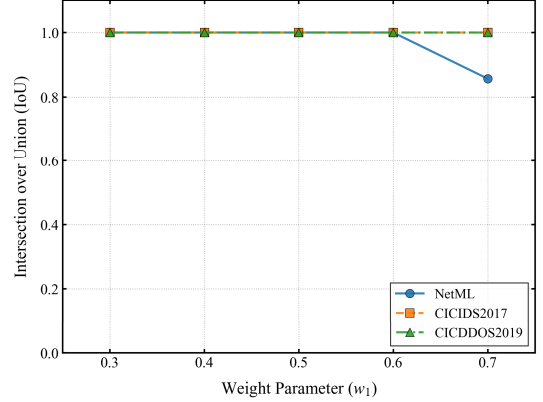


Fig. 9. Sensitivity analysis of w_1 (IoU with baseline $w_1 = 0.5$).

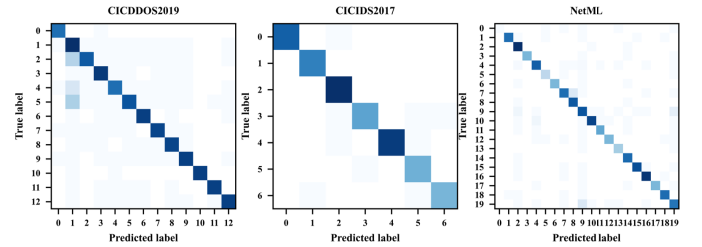


Fig. 10. Confusion matrix for RET-Net.

early payloads rather than standard initial headers; and (2) in heterogeneous traffic environments where diverse protocols render a fixed extraction window suboptimal.

G. Compare RET-Net to Baseline Models

In this phase of the experiment, it was ensured that each model was trained on an identical total byte count across the same datasets. Furthermore, the baseline models were set to extract byte sequences starting from the zeroth byte of each packet by default. Fig. 10 displays the performance of the RET-Net model through the confusion matrix on three datasets. The comparison with multiple baseline models, as shown in Figs. 11~14, demonstrates that RET-Net exhibited superior classification performance on all three datasets. Particularly, on the CICIDS2017 dataset, RET-Net's classification ability significantly surpassed other models. This result confirms both the strong traffic classification capacity of our lightweight CNN model and the effectiveness of the Key-

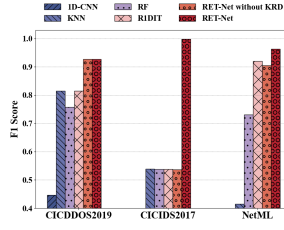


Fig. 11. F1 score of all models.

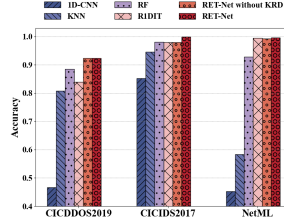


Fig. 12. Accuracy of all models.

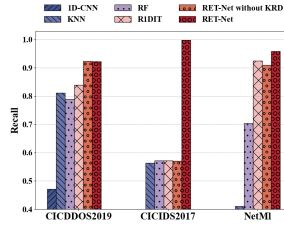


Fig. 13. Recall of all models.

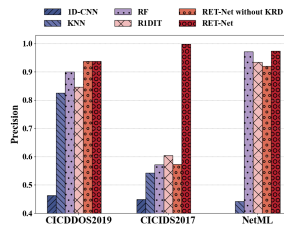


Fig. 14. Precision of all models.

Byte mechanism in enhancing classification accuracy without augmenting the input data volume.

On the NetML, we conducted a temporal performance analysis to compare the serial and parallel classification latencies of the RET-Net model with other models under identical input data conditions. Serial classification latency reflects the model's potential adverse impact on the normal communication of network flows. In contrast, parallel classification latency assesses the model's concurrency capabilities.

As shown in table VI, in a serial setup, RET-Net's classification latency is a mere 0.201 milliseconds. Compared to RIDIT, which employs a self-attention mechanism, RET-Net demonstrates a remarkable 21.52-fold increase in classification speed. This significant enhancement in speed is attributed to RET-Net's unique design, where we have decoupled the complex self-attention mechanism from the model through a key-byte mechanism and incorporated a lightweight Con-

TABLE VI
TEMPORAL PERFORMANCE COMPARISON

Model	Input	Serial	Parallel
RET-Net	2x30	0.201ms	22 μ s
1D-CNN	2x30	0.117ms	17 μ s
1D-CNN	7x112	0.841ms	116 μ s
KNN	2x30	4.247ms	149 μ s
RF	2x30	3.402ms	5 μ s
RIDIT	2x30	4.326ms	46 μ s

TABLE VII
RF RESULTS ON NetML UNDER SERIAL EXECUTION (max_depth = 20)

$n_{\text{estimators}}$	Macro-F1 score	Serial inference latency (ms)
50	0.8383	1.4016
100	0.8914	3.4024
150	0.8975	5.3432
200	0.8953	6.3090
250	0.8929	7.8816

volutional Neural Network. Although RET-Net is slightly slower by a factor of 1.72 when compared to 1D-CNN, the results depicted in Fig 11 indicate that 1D-CNN significantly falls behind RET-Net in terms of classification accuracy. This implies that 1D-CNN's rapid classification speed does not translate into practical benefits due to the sacrifice of accuracy. Moreover, RET-Net has achieved improvements in both classification speed and accuracy when compared to machine learning algorithms such as KNN and RF.

We additionally evaluated Random Forest on NetML under serial execution with max_depth = 20 and $n_{\text{estimators}} \in \{50, 100, 150, 200, 250\}$. As shown in Table VII, the Macro-F1 score increases from 0.8383 at 50 estimators to 0.8914 at 100, and reaches 0.8975 at 150. When the ensemble size is further increased to 200 and 250, the Macro-F1 score slightly decreases to 0.8953 and 0.8929, respectively. In contrast, the serial inference latency increases steadily from 1.4016 ms to 3.4024, 5.3432, 6.3090, and 7.8816 ms. These results suggest that a larger RF model can improve accuracy to some extent, but the gain becomes limited once the ensemble is already moderately large, while the temporal cost continues to increase. Overall, RET-Net still provides a more favorable balance between classification accuracy and serial inference latency for real-time traffic classification.

Additionally, we analyzed the serial latency under different input conditions. Fig 15 illustrates the trend of RET-Net's classification latency as it varies with the number of input bytes. It is observable that there is a positive correlation between RET-Net's classification latency and the number of input bytes. As the number of input bytes decreases, RET-Net's classification latency exhibits an approximately linear decrease, indicating that reducing the number of input bytes yields stable classification speedup benefits.

In parallel settings, we utilize batch size to define the number of flows processed simultaneously by the model. It should be noted that this batch size setup is strictly designed for the inference phase, rather than the training phase. Specifically, the batch sizes adopted in our parallel inference setup are 4, 8, 16,

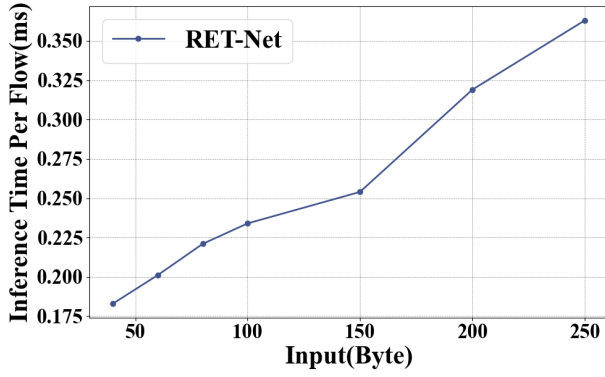


Fig. 15. Inference time under varying input conditions in the serial setting.

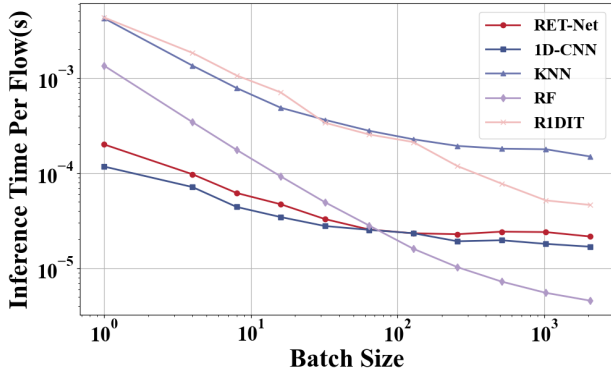


Fig. 16. Inference time under varying batch size in the parallel setting.

32, 64, 128, 256, 512, 1024, and 2048. Furthermore, to ensure a fair comparison of parallel inference efficiency, this identical batch processing configuration is also applied to the traditional machine learning models, including RF and KNN. Fig. 16 clearly illustrates how the inference time of different models varies with changes in batch size. At smaller batch sizes, specifically less than 64, RET-Net outperforms the RF, KNN, and RIDIT models. When the batch size increases beyond 64, although RET-Net is slightly behind RF and 1D-CNN, it still outperforms RIDIT and KNN. As the batch size grows, RET-Net's average inference time shows a trend of stabilization, ultimately settling at $22\mu s$. This result indicates that RET-Net can handle large-scale network flow classification tasks with exceptional efficiency.

H. Incremental Learning Analysis

1) *Experimental Scenarios*: To further evaluate the robustness of our incremental learning mechanism against catastrophic forgetting, we expanded the experimental scope beyond a single configuration. We defined three distinct scenarios using the NetML and CICDDOS2019 datasets, varying the initial knowledge base size, incremental step size, and traffic distribution. In all scenarios, λ is equal to 0.1.

- **NetML (15 + 1 Scenario)**: This setting serves as the baseline evaluation for fine-grained classification. The model is initialized with 15 base classes, and new traffic is introduced sequentially, adding 1 class per phase over

TABLE VIII
MACRO-F1 SCORES AND RETENTION RATES ACROSS INCREMENTAL PHASES (EXEMPLARS/CLASS = 400). VALUES IN PARENTHESES INDICATE THE PERCENTAGE DROP RELATIVE TO PHASE 0

Incremental Phase	NetML		CICDDOS2019
	10+2 Scenario	15+1 Scenario	8+1 Scenario
Phase 0 (Base)	0.9951	0.9887	0.9445
Phase 1	0.9198 (-7.6%)	0.9684 (-2.1%)	0.9421 (-0.3%)
Phase 2	0.8092 (-18.7%)	0.9513 (-3.8%)	0.8763 (-7.2%)
Phase 3	0.7335 (-26.3%)	0.9059 (-8.4%)	0.7754 (-17.9%)
Phase 4	0.6717 (-32.5%)	0.8732 (-11.7%)	0.7823 (-17.2%)
Phase 5 (Final)	0.6238 (-37.3%)	0.7916 (-19.9%)	0.8033 (-15.0%)

a total of 6 phases. This evaluates the model's stability under sequential update conditions.

- **NetML (10+2 Scenario)**: This represents a scenario with larger incremental steps. The model starts with a reduced knowledge base of 10 classes and is updated with two new classes per phase. This setting evaluates the capacity to handle larger batches of new information relative to a smaller existing knowledge base.
- **CICDDOS2019 (8 + 1 Scenario)**: To examine generalizability across datasets, we incorporated the CICDDOS2019 dataset. Initialized with 8 base classes, the model incrementally learns 1 new DDoS attack type per phase. Unlike NetML, this dataset contains classes with highly correlated structural patterns, testing whether RET-Net can preserve discriminative boundaries in homogeneous traffic environments.

2) *Robustness Evaluation*: To quantify the model's resistance to catastrophic forgetting, we analyzed the performance trajectories across the defined scenarios. Fig. 17 illustrates the variations in the Macro-F1 score across sequential incremental phases for different exemplar memory sizes (Exemplars/Class ranging from 50 to 500). Furthermore, Table VIII presents the specific Macro-F1 scores and retention rates for the setting with 400 exemplars per class, which was empirically selected as an empirically balance between memory constraints and classification accuracy.

Early-Phase Stability (Phases 1–2): In the initial incremental stages, defined as Phases 1 and 2, RET-Net demonstrates strong robustness. The NetML 15 + 1 scenario exemplifies this stability, where the model maintains high classification performance with a Macro-F1 score of 0.9513 at Phase 2, representing a retention rate exceeding 96% relative to the baseline. Similarly, the CICDDOS2019 scenario sustains an F1 score of 0.8763 during this period. These results indicate that for moderate traffic variations, the initial Key-Byte regions remain valid, and the model effectively assimilates new knowledge through parameter adaptation without requiring structural reconfiguration.

Analysis of Late-Phase Degradation (Phases 3–5): Contrastingly, a noticeable performance degradation emerges during the late stages, spanning Phases 3 through 5. This trend is particularly pronounced in the more challenging NetML 10 + 2 scenario, where the F1 score declines to 0.6238 by the final phase. We attribute this degradation to two factors:

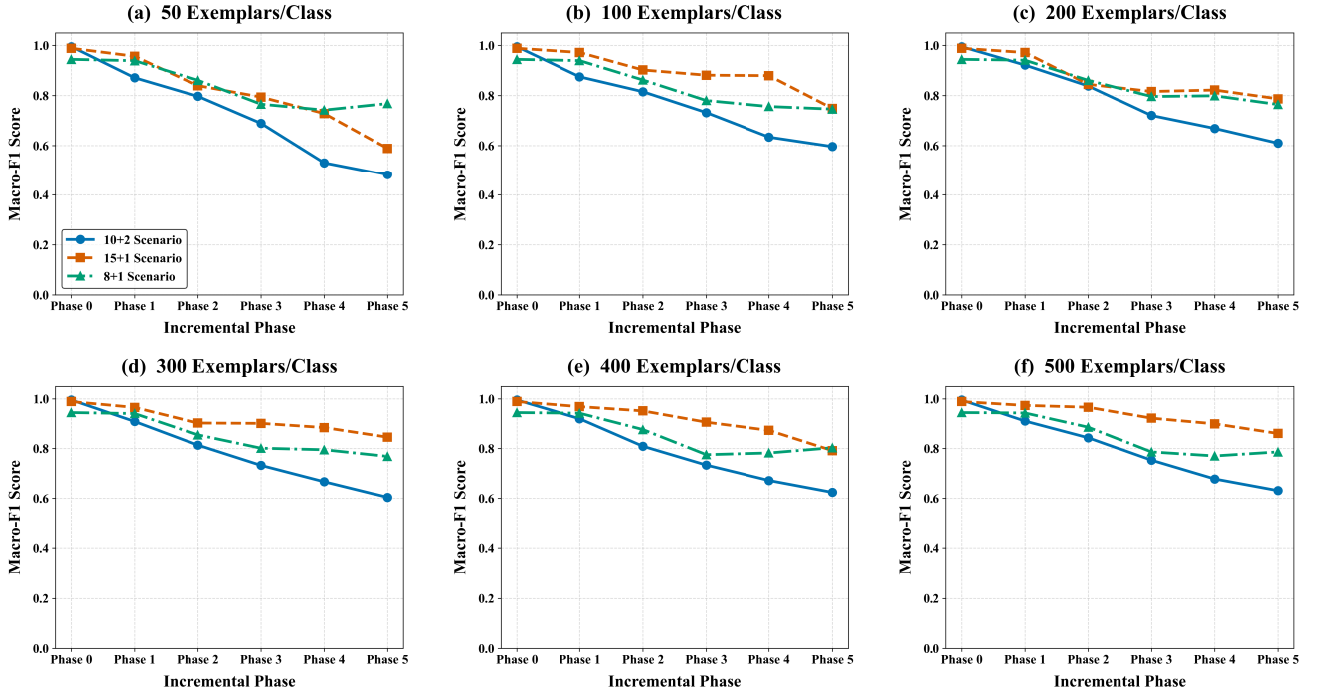


Fig. 17. Macro-F1 Score variations across incremental phases for different exemplar sizes (50 to 500) and scenarios on NetML and CICDDOS2019 datasets.

- 1) **Catastrophic Forgetting:** As the sequence of tasks grows, the model's capacity to retain boundaries for original classes naturally diminishes, a phenomenon partially mitigated by our exemplar replay and knowledge distillation mechanisms.
- 2) **Distributional Shift in Key Features:** More critically, as new diverse traffic types are introduced, their discriminative features may spatially lie outside the fixed Key-Byte region optimized for the initial base classes. This spatial misalignment significantly restricts the model from accessing the necessary information, regardless of parameter fine-tuning.

Adaptive KRD Update Strategy: Based on the analysis of performance degradation, we propose a potential maintenance strategy to balance the trade-off between adaptability and computational cost:

- **Tier 1 (Soft Update):** In scenarios where performance remains stable, standard incremental learning appears sufficient. This is supported by the observations in the first four phases of the NetML 15 + 1 scenario, where the existing Key-Byte region remains valid. In such cases, avoiding structural updates minimizes computational overhead.
- **Tier 2 (Hard Update):** Conversely, a significant drop in metrics could signal a critical feature shift, as seen in Phase 3 of the NetML 10 + 2 scenario. This phenomenon suggests the necessity of re-executing the KRD algorithm to discover an updated, representative Key-Byte region that covers the novel traffic patterns.

In addition to performance-driven triggers, practical deployments could leverage off-peak periods (e.g., nighttime) to perform scheduled Hard Updates. This proactive measure

helps sustain the model's robust feature extraction capability without disrupting real-time services during high-load periods, thereby maintaining high classification efficacy over long-term operation.

VI. DISCUSSION

RET-Net is inherently protocol-agnostic and adaptive. By processing packets as raw byte sequences rather than relying on manual feature engineering, the framework generalizes to various protocols without requiring redesign of the core neural network architecture. A critical practical advantage of this data-driven design is its robust generalization capability across heterogeneous traffic distributions. Although key-byte regions exhibit dataset-specific variations, their overlap confirms that they capture deterministic protocol header fields rather than arbitrary statistical artifacts. By aggregating feature importance across diverse traffic classes, the KRD algorithm inherently identifies a broadly applicable region. Consequently, RET-Net functions reliably in heterogeneous networks without requiring prior knowledge of traffic distribution.

However, the Key-Byte mechanism exhibits operational boundaries in extreme scenarios. If a network environment contains fundamentally different protocols with completely non-overlapping key-byte locations, KRD effectively degrades into a union strategy to cover all types. This significantly enlarges the selected region, increasing RET-Net's input data size and diminishing its speed advantage over traditional deep learning models. Additionally, if traffic patterns evolve abruptly or if adversarial attacks are specifically crafted to manipulate the identified key bytes, the existing region may become invalid, necessitating a re-execution of

the KRD algorithm and incurring additional computational overhead.

Moving forward, deep learning-based traffic classification faces ongoing challenges from encryption evolution and interpretability limits. The widespread adoption of advanced cryptographic standards such as TLS 1.3 progressively diminishes available plaintext metadata, making exclusive reliance on spatial byte features insufficient. Furthermore, while our post-hoc mechanisms, specifically Grad-CAM and KRD, effectively locate discriminative features, they cannot fully elucidate the underlying non-linear combinatorial logic of the model. Acknowledging these boundaries, our future research will focus on evolving RET-Net into a multi-modal framework. By integrating protocol-agnostic side-channel information such as packet timing, we aim to sustain classification efficacy in zero-visibility networks while concurrently exploring advanced analytical approaches to further improve model interpretability for network operators.

VII. CONCLUSION

In this paper, we introduce a novel real-time traffic classification framework, RET-Net. RET-Net improves classification accuracy and speed by precisely locating key bytes in packets via the Key-Byte mechanism. Then the RET-Net model employs a lightweight CNN for classification. Our experimental evaluations on 3 different datasets demonstrate the superior performance of RET-Net, with F1 scores ranging from 0.9273 to 0.9983, which significantly outperforms existing high-level models. A key highlight of RET-Net is its speed. More specifically, it processes each flow in just 0.201ms when executed serially, marking a significant 21.52-fold speed enhancement over R1DIT. This high-speed characteristic positions RET-Net as a viable solution for the majority of internet services. Meanwhile, RET-Net supports incremental learning by constructing an exemplars set with the herding algorithm and combining it with knowledge distillation. By sacrificing a small amount of old knowledge, the model can efficiently adapt to new traffic types. RET-Net has been successfully integrated into TADK [57] and it is available for public use.

REFERENCES

- [1] S. Rezaei and X. Liu, "Deep learning for encrypted traffic classification: An overview," *IEEE Commun. Mag.*, vol. 57, no. 5, pp. 76–81, May 2019.
- [2] A. A. Ahmed and G. Agunsoye, "A real-time network traffic classifier for online applications using machine learning," *Algorithms*, vol. 14, no. 8, p. 250, Aug. 2021.
- [3] M. Camelo, P. Soto, and S. Latré, "A general approach for traffic classification in wireless networks using deep learning," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 4, pp. 5044–5063, Dec. 2022.
- [4] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE Conf. Comput. Commun. (IEEE INFOCOM)*, Apr. 2019, pp. 1171–1179.
- [5] Y. Zeng, H. Gu, W. Wei, and Y. Guo, "Deep-full-range: A deep learning based network encrypted traffic classification and intrusion detection framework," *IEEE Access*, vol. 7, pp. 45182–45190, 2019.
- [6] O. Barut, Y. Luo, P. Li, and T. Zhang, "R1DIT: Privacy-preserving malware traffic classification with attention-based neural networks," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 2, pp. 2071–2085, Jun. 2023.
- [7] H. A. Jamil, B. M. Ali, M. Hamdan, and A. E. Osman, "Online P2P internet traffic classification and mitigation based on snort and ML," *Eur. J. Eng. Res. Sci.*, vol. 4, no. 10, pp. 131–137, Oct. 2019.
- [8] G. Perna et al., "Online classification of RTC traffic," in *Proc. IEEE 18th Annu. Consum. Commun. Netw. Conf. (CCNC)*, Jan. 2021, pp. 1–6.
- [9] P. Tang, Y. Dong, S. Mao, H.-L. Wei, and J. Jin, "Online classification of network traffic based on granular computing," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 53, no. 8, pp. 5199–5211, Aug. 2023.
- [10] M. Lotfollahi, M. J. Siavoshani, R. S. H. Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Comput.*, vol. 24, no. 3, pp. 1999–2012, Feb. 2020.
- [11] D. Joon and M. Pundir, "A comprehensive investigation into the implementation of machine learning solutions for network traffic classification," in *Proc. Int. Conf. Adv. Comput. Commun. Technol. (ICACCTech)*, Dec. 2023, pp. 467–472.
- [12] G. Aceto, D. Ciunzo, A. Montieri, and A. Pescapé, "Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges," *IEEE Trans. Netw. Service Manage.*, vol. 16, no. 2, pp. 445–458, Jun. 2019.
- [13] P. Wang, X. Chen, F. Ye, and Z. Sun, "A survey of techniques for mobile service encrypted traffic classification using deep learning," *IEEE Access*, vol. 7, pp. 54024–54033, 2019.
- [14] C. Li, C. Dong, K. Niu, and Z. Zhang, "Mobile service traffic classification based on joint deep learning with attention mechanism," *IEEE Access*, vol. 9, pp. 74729–74738, 2021.
- [15] H. Huang, H. Deng, J. Chen, L. Han, and W. Wang, "Automatic multi-task learning system for abnormal network traffic detection," *Int. J. Emerg. Technol. Learn. (iJET)*, vol. 13, no. 4, p. 4, Mar. 2018.
- [16] G. Aceto, D. Ciunzo, A. Montieri, and A. Pescapé, "Mobile encrypted traffic classification using deep learning," in *Proc. Netw. Traffic Meas. Anal. Conf. (TMA)*, Jun. 2018, pp. 1–8.
- [17] G. Bovenzi, G. Aceto, D. Ciunzo, V. Persico, and A. Pescapé, "A hierarchical hybrid intrusion detection approach in IoT scenarios," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2020, pp. 1–7.
- [18] L. Mostacero-Agama and P. Shiguihara, "Analysis of internet service latency and its impact on Internet of Things (IoT) applications," in *Proc. IEEE Eng. Int. Res. Conf. (EIRCON)*, Oct. 2022, pp. 1–4.
- [19] I. Parvez, A. Rahmati, I. Guvenc, A. I. Sarwat, and H. Dai, "A survey on low latency towards 5G: RAN, core network and caching solutions," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 4, pp. 3098–3130, 2018.
- [20] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, "ICaRL: Incremental classifier and representation learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2001–2010.
- [21] A. Dhakad, S. Singh, Mohana, M. Moharir, and A. R. A. Kumar, "Real time network traffic analysis using artificial intelligence, machine learning and deep learning: A review of methods, tools and applications," in *Proc. Int. Conf. Self Sustain. Artif. Intell. Syst. (ICSSAS)*, Oct. 2023, pp. 372–378.
- [22] A. Azab, M. Khasawneh, S. Alrabae, K.-K.-R. Choo, and M. Sarsour, "Network traffic classification: Techniques, datasets, and challenges," *Digit. Commun. Netw.*, vol. 10, no. 3, pp. 676–692, Jun. 2024.
- [23] Y. Qi, L. Xu, B. Yang, Y. Xue, and J. Li, "Packet classification algorithms: From theory to practice," in *Proc. IEEE INFOCOM*, Apr. 2009, pp. 648–656.
- [24] R. M. AlZoman and M. J. F. Alenazi, "A comparative study of traffic classification techniques for smart city networks," *Sensors*, vol. 21, no. 14, p. 4677, Jul. 2021.
- [25] J. Zhao, X. Jing, Z. Yan, and W. Pedrycz, "Network traffic classification for data fusion: A survey," *Inf. Fusion*, vol. 72, pp. 22–47, Aug. 2021.
- [26] P. Choorod, G. Weir, and A. Fernando, "Classifying tor traffic encrypted payload using machine learning," *IEEE Access*, vol. 12, pp. 19418–19431, 2024.
- [27] K. L. Dias, M. A. Pongelupe, W. M. Caminhas, and L. de Errico, "An innovative approach for real-time network traffic classification," *Comput. Netw.*, vol. 158, pp. 143–157, Jul. 2019.
- [28] S. Ahn, J. Kim, S. Y. Park, and S. Cho, "Explaining deep learning-based traffic classification using a genetic algorithm," *IEEE Access*, vol. 9, pp. 4738–4751, 2021.
- [29] S. Dong, "Multi class SVM algorithm with active learning for network traffic classification," *Expert Syst. Appl.*, vol. 176, Aug. 2021, Art. no. 114885.
- [30] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2018, pp. 1928–1943.

- [31] L. D. Manocchio, S. Layeghy, W. W. Lo, G. K. Kulatilleke, M. Sarhan, and M. Portmann, "FlowTransformer: A transformer framework for flow-based network intrusion detection systems," *Expert Syst. Appl.*, vol. 241, May 2024, Art. no. 122564.
- [32] K. Lin, X. Xu, and H. Gao, "TSCRNN: A novel classification scheme of encrypted traffic based on flow spatiotemporal features for efficient management of IIoT," *Comput. Netw.*, vol. 190, May 2021, Art. no. 107974.
- [33] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.
- [34] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, Apr. 2022, pp. 633–642.
- [35] T.-D. Pham, T.-L. Ho, T. Truong-Huu, T.-D. Cao, and H.-L. Truong, "MAppGraph: Mobile-app classification on encrypted network traffic using deep graph convolution neural networks," in *Proc. Annu. Comput. Secur. Appl. Conf.*, Dec. 2021, pp. 1025–1038.
- [36] H. Huang, Y. Zhou, and F. Jiang, "CLA-BERT: A hybrid model for accurate encrypted traffic classification by combining packet and byte-level features," *Mathematics*, vol. 13, no. 6, p. 973, Mar. 2025.
- [37] R. Masukawa et al., "PACKETCLIP: Multi-modal embedding of network traffic and language for cybersecurity reasoning," *Frontiers Artif. Intell.*, vol. 8, Jul. 2025, Art. no. 1593944.
- [38] L. Liu, R. Li, Q. Li, M. Hou, Y. Jiang, and M. Xu, "FlowletFormer: Network behavioral semantic aware pre-training model for traffic classification," 2025, *arXiv:2508.19924*.
- [39] G. Aceto, D. Ciunzo, A. Montieri, and A. Pescapè, "MIMETIC: Mobile encrypted traffic classification using multimodal deep learning," *Comput. Netw.*, vol. 165, Dec. 2019, Art. no. 106944.
- [40] M. Xiao, Y. Li, Y. Zhao, T. Guan, M. Xia, and W. Li, "Synecdoche: Efficient and accurate in-network traffic classification via direct packet sequential pattern matching," 2025, *arXiv:2512.21116*.
- [41] H. Y. He, Z. Guo Yang, and X. N. Chen, "PERT: Payload encoding representation from transformer for encrypted traffic classification," in *Proc. ITU Kaleidoscope, Ind.-Driven Digit. Transformation (ITU K)*, Dec. 2020, pp. 1–8.
- [42] P. Lin, K. Ye, Y. Hu, Y. Lin, and C.-Z. Xu, "A novel multimodal deep learning framework for encrypted traffic classification," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1369–1384, Jun. 2023.
- [43] Z. Sha, C. Sun, S. Wei, R. Huo, and H. Ren, "A transformer based classified traffic prediction scheme for energy digital twin network," in *Proc. IEEE Int. Conf. Energy Internet (ICEI)*, Oct. 2023, pp. 218–223.
- [44] L. Yu et al., "PBCNN: Packet bytes-based convolutional neural network for network intrusion detection," *Comput. Netw.*, vol. 194, Jul. 2021, Art. no. 108117.
- [45] W. Zhu, X. Ma, Y. Jin, and R. Wang, "ILETC: Incremental learning for encrypted traffic classification using generative replay and exemplar," *Comput. Netw.*, vol. 224, Apr. 2023, Art. no. 109602.
- [46] F. Cerasuolo et al., "MEMENTO: A novel approach for class incremental learning of encrypted traffic," *Comput. Netw.*, vol. 245, Mar. 2024, Art. no. 110374.
- [47] Z. Li, L. Bu, Y. Wang, Q. Ma, L. Tan, and F. Bu, "Hierarchical perception for encrypted traffic classification via class incremental learning," *Comput. Secur.*, vol. 149, Feb. 2025, Art. no. 104195.
- [48] D. Di Monda, A. Montieri, V. Persico, P. Voria, M. De Ieso, and A. Pescapè, "Few-shot class-incremental learning for network intrusion detection systems," *IEEE Open J. Commun. Soc.*, vol. 5, pp. 6736–6757, 2024.
- [49] Y. Chen, T. Zang, Y. Zhang, Y. Zhou, L. Ouyang, and P. Yang, "Incremental learning for mobile encrypted traffic classification," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Jun. 2021, pp. 1–6.
- [50] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 618–626.
- [51] R. R. Selvaraju, A. Das, R. Vedantam, M. Cogswell, D. Parikh, and D. Batra, "Grad-CAM: Why did you say that?," 2016, *arXiv:1611.07450*.
- [52] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.
- [53] O. Barut, Y. Luo, T. Zhang, W. Li, and P. Li, "NetML: A challenge for network traffic analytics," 2020, *arXiv:2004.13006*.
- [54] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy," in *Proc. Int. Carnahan Conf. Security Technol. (ICCSST)*, 2019, pp. 1–8.
- [55] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy*, 2018, pp. 108–116.
- [56] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *Proc. IEEE Int. Conf. Intell. Secur. Informat. (ISI)*, Jul. 2017, pp. 43–48.
- [57] K. Qiu et al., "Traffic analytics development kits (TADK): Enable real-time AI inference in networking apps," in *Proc. 13th Int. Conf. Ubiquitous Future Netw. (ICUFN)*, Jul. 2022, pp. 392–398.